Learning the Tracker from Data: Bayesian Decision and Maximum-Likelihood Methods for Camera-Based Target Tracking on a Drone and an Unmanned Surface Vessel

Pattern Recognition, Department of Electrical Engineering, National Taipei University of Technology
Final project: midterm proposal (Question 1) and proposed method for the final presentation
Group members: _____ (USV platform) · _____ (drone platform) · 4 November 2026

Summary. Both members of our group track one target with a camera, using the same pipeline: YOLOv8 finds the target, Lucas-Kanade optical flow moves the box between detections, and a Kalman filter combines them. One platform is a drone; the other is a fleet of unmanned surface vessels (USVs) that keep formation by following a leader boat. Both break down when the target's box moves up and down quickly in the image. On the drone the target climbs and drops; on the USV the camera itself pitches in waves. The current tracker depends on hand-tuned constants. We replace them with course methods: **maximum-likelihood estimation** (Ch. 3-1) of the filter's noise, a **minimum-risk Bayes rule** (Ch. 2) for accepting or rejecting each detection, and **PCA, Fisher LDA and the Hughes effect** (Ch. 3-2, Ch. 1) for choosing features. A first offline test already shows the main effect. Choosing the process noise by maximum likelihood turns the drone tracker from worse than raw YOLO on the boat (73% vs 85% of frames tracked well in clear water) into better than it (88%). On the drone it lifts accuracy from 47–57% to 72–82% when the detector is slow.

Part I. Midterm proposal (Question 1)

1. Topic

Problem. A camera tracker has to decide two things in every frame: *which* detected box is the target, and *how much* to trust it compared with its own prediction. In our current code, both decisions use hand-tuned constants: a distance gate (`GATE = 1.5`), a size-ratio window (0.25 to 5), measurement noise $R = 60 \cdot (0.5/\text{confidence})^2$, and a fixed process noise Q that was "raised by hand so the filter keeps up when the drone accelerates". These values were tuned for a 30 fps, 1080p drone video. They have no basis in data and do not carry over to another platform.

Shared difficulty. Fast vertical motion of the target's box. On our USV logs, the share of frames where YOLO misses the leader grows from 1.8% to 14.7% as the box's vertical speed rises from 0–2 to 20–40 px per step. The share where YOLO labels the wrong box as the leader grows from 0.5% to 7.6% (Figure 1).

Idea. Treat each hand-tuned decision as a pattern recognition problem and learn it from labelled data:

Hand-tuned part	Replacement from the course	Chapter
Measurement noise R	Gaussian maximum-likelihood estimate of the detection error covariance, per axis and per confidence level	Ch. 3-1
Process noise Q	Maximum-likelihood estimate: closed form from ground-truth trajectories, or by maximising the likelihood of the filter's prediction errors	Ch. 3-1
Gate and ranking score	Minimum-risk Bayes classifier (target, other object, clutter) with Gaussian class-conditional densities estimated by maximum likelihood	Ch. 2, 3-1
Choice of features	PCA, Fisher LDA projection and an experimental Hughes curve (accuracy against number of features and training-set size)	Ch. 3-2, Ch. 1
Evaluation	Design cycle, ROC curves, held-out test data	Ch. 1

2. Related papers

1. R. O. Duda, P. E. Hart, D. G. Stork, *Pattern Classification*, 2nd ed., Wiley, 2001. Course textbook: Bayes decision theory, MLE, PCA and LDA.
2. G. F. Hughes, "On the mean accuracy of statistical pattern recognizers," *IEEE Trans. Inf. Theory*, IT-14, 1968. Accuracy peaks, then falls, as dimension grows.
3. R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of Eugenics*, 1936. Fisher's linear discriminant.
4. R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Engineering*, 1960. The Kalman filter as recursive Bayesian estimation.
5. R. K. Mehra, "On the identification of variances and adaptive Kalman filtering," *IEEE Trans. Automatic Control*, 1970. Maximum-likelihood estimation of Q and R from innovations.
6. B. D. Lucas, T. Kanade, "An iterative image registration technique with an application to stereo vision," IJCAI 1981. The optical flow used on both platforms.
7. Z. Kalal, K. Mikolajczyk, J. Matas, "Forward-backward error: automatic detection of tracking failures," ICPR 2010. The flow check in our tracker.
8. Y. Bar-Shalom, E. Tse, "Tracking in a cluttered environment with probabilistic data association," *Automatica*, 1975. Validation gates under clutter.
9. H. A. P. Blom, Y. Bar-Shalom, "The interacting multiple model algorithm for systems with Markovian switching coefficients," *IEEE Trans. Automatic Control*, 1988. Switching between motion models.
10. A. Bewley et al., "Simple online and realtime tracking," ICIP 2016. A detector combined with a Kalman filter and an association step.
11. C. Guo et al., "On calibration of modern neural networks," ICML 2017. A CNN's confidence score is not a calibrated posterior probability.

3. Project tasks and schedule

#	Task	Work	Status
T1	Labelled data	USV: 652 logged runs with exact simulator ground truth. Drone: Grounding DINO labels for two test videos, checked by eye.	Done
T2	Detection errors against motion	Missed and wrong boxes against the target's vertical image speed, on both platforms	USV done; drone first pass
T3	MLE of R	Error covariance per axis and confidence level; check the Gaussian assumption with Q-Q plots	Done; Q-Q to do
T4	MLE of Q	(a) closed form from ground truth (textbook); (b) innovation likelihood (no ground truth needed)	(b) done; (a) to do
T5	Bayes gate	Minimum-risk rule; sweep the loss ratio $\lambda_{FA}/\lambda_{miss}$; ROC curves; re-acquire the target after a run of rejections	First version done
T6	PCA, LDA, Hughes	2-D Fisher projection of the three classes; PCA explained variance; accuracy against number of features d for n = 20 to 10,000 training samples	To do
T7	Tracking replay	All tracker versions on both platforms, scored against ground truth	Done
T8	Closed loop (optional)	Learned tracker inside the USV formation simulator, huge-wave laps; live drone test if time allows	Optional

Week	Dates	Work
W1	Nov 5 – Nov 11	Proposal; 3-minute proposal talk (Nov 11)
W2	Nov 12 – Nov 18	T4(a) closed-form Q; T3 Q-Q plots; a latency-aware R fit
W3	Nov 19 – Nov 25	T5 loss sweep, ROC, re-acquisition
W4	Nov 26 – Dec 2	T6 PCA, LDA, Hughes curve
W5	Dec 3 – Dec 9	T2 drone motion analysis with smoothed ground truth; more drone test videos
W6	Dec 10 – Dec 16	T8 optional closed-loop runs
W7	Dec 17 – Dec 23	Report, figures, comparison videos
W8	Dec 24 → final	Rehearsal; spare time

The final presentation date is still to be confirmed; we will move the last two weeks to fit it.

4. Final product

1. **A shared Python module** with three parts: a Bayes gate, a noise model (R and Q learned by MLE), and a feature analysis (PCA and LDA). It replaces the hand-tuned constants in both trackers.
2. **Two labelled datasets**: USV candidate boxes with exact ground truth in six weather conditions, and drone videos with checked pseudo ground truth.
3. **A reproducible evaluation**: a deterministic replay of the tracker on both platforms, with all figures and tables in this report.
4. **Side-by-side videos**: YOLO only against the learned tracker, for the drone and for a huge-wave boat run.

5. Project report: components and items

Abstract; introduction (fast vertical image motion: a moving target on the drone, a moving camera on the USV); related work; problem formulation (classes, feature vector, loss matrix, the minimum-risk rule derived as a likelihood-ratio inequality); method (MLE of R and Q with derivations, Bayes gate, PCA and LDA); data and ground truth; results on each platform, then compared; discussion (when the Gaussian assumption fails, dimension against sample size, limitations); conclusion and future work; references; appendix with code and reproduction steps.

6. Project presentation (about 15 minutes, both members)

Motivation with two video clips, drone and boat (2 min) → the shared pipeline and its hand-tuned parts (2 min) → the pattern recognition formulation linked to the lecture slides (3 min) → USV results (3 min) → drone results (3 min) → cross-platform conclusion and questions (2 min). Details are in Part IV.

Part II. Proposed method

II.1 System and notation

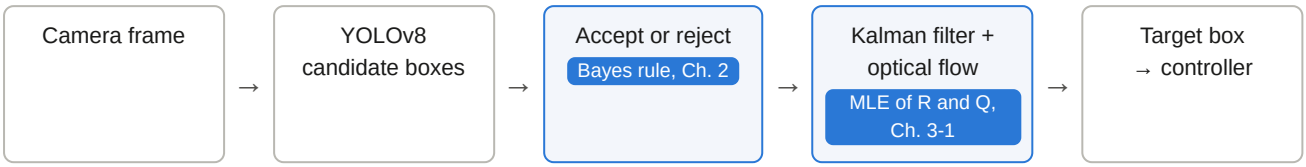


Figure 0. The shared tracker. Blue boxes are where the course methods replace hand-tuned constants.

The filter state is $x = (c_x, c_y, w, h, v_x, v_y, v_w, v_h)^t$, with a constant-velocity transition F. A measurement $z = Hx + v$ (a YOLO box or a flow-shifted box) has noise $v \sim N(0, R)$. The motion model is $x_{k+1} = Fx_k + w_k$, with $w_k \sim N(0, Q)$. Errors are measured in units of box size, so one model works for both near and far targets.

II.2 Method 1: maximum-likelihood estimate of the measurement noise R (Ch. 3-1)

For each training detection that matches the true box, the error is $e_k = (z_k - z_k^{true})/\text{size}$. With $e \sim N(\mu, \Sigma)$, the slides' Gaussian case with unknown μ and Σ gives the closed-form MLE:

$$\hat{\mu} = (1/n)\,\Sigma_k\, e_k, \quad \hat{\Sigma} = (1/n)\,\Sigma_k\,(e_k - \hat{\mu})(e_k - \hat{\mu})^t$$

We fit $\hat{\Sigma}$ separately for the horizontal and vertical axes and for each YOLO-confidence bin. This tests the drone code's assumption that a single variance, scaled by confidence, holds on both axes.

II.3 Method 2: maximum-likelihood estimate of the process noise Q (Ch. 3-1)

(a) With ground truth: closed form, as in the textbook. From the true box trajectory, the process residuals are $w_k = x_{k+1} - Fx_k$, and the MLE is $\hat{Q} = (1/n)\,\Sigma_k\, w_k w_k^t$. This is the same derivation as for R, applied to the motion instead of the measurement.

(b) Without ground truth: innovation likelihood (Mehra, 1970). For a correct model, the innovations $\nu_k = z_k - H\hat{x}_{k|k-1}$ are independent, with $\nu_k \sim N(0, S_k)$, where $S_k = HP_{k|k-1}H^t + R$. So we choose Q to maximise

$$\ell(Q) = \Sigma_k \ln N(\nu_k; 0, S_k(Q)) = -\tfrac{1}{2}\,\Sigma_k\,[\,\nu_k^t S_k^{-1}\nu_k + \ln|S_k| + d\ln 2\pi\,]$$

This is the Ch. 3-1 principle: the observed samples are the innovations, and Q is the parameter. Because S_k depends on Q through the filter's recursion, there is no closed-form solution, so we maximise over a grid. Method (b) is what runs on a real drone or boat. Method (a) is the textbook check when ground truth exists. The final presentation will compare the two.

II.4 Method 3: minimum-risk Bayes rule for accepting a detection (Ch. 2)

Each candidate box gives a feature vector **x**: YOLO confidence, Mahalanobis distance to the predicted box under the filter's covariance, log area ratio and log aspect ratio. Classes: ω_1 = target, ω_2 = not the target. λ_{ij} is the loss of deciding ω_i when the truth is ω_j. Accepting a false box (λ_{12}) costs more than rejecting a true one (λ_{21}), because the filter can coast on its prediction for a few frames. Minimising the conditional risk $R(\alpha_i|x) = \Sigma_j\,\lambda_{ij}P(\omega_j|x)$ gives the rule: accept if

$$p(x|\omega_1)\,/\,p(x|\omega_2) > [(\lambda_{12} - \lambda_{22})\,/\,(\lambda_{21} - \lambda_{11})]\cdot P(\omega_2)\,/\,P(\omega_1)$$

The class-conditional densities $p(x|\omega_i)$ are Gaussians fitted by MLE, as in Method 1. The priors $P(\omega_i)$ are estimated per platform and per condition. Distance is measured with the filter's own covariance, so the acceptance region widens automatically while the tracker coasts. This replaces the hand-tuned gate growth of (1 + 0.15·misses).

II.5 Method 4: feature analysis with PCA, Fisher LDA and the Hughes effect (Ch. 3-2, Ch. 1)

Project the candidate features of the three classes (target, other object, clutter) onto the 2-D Fisher subspace (c − 1 = 2 dimensions) for each condition, and show PCA's explained variance. Then measure Bayes-gate accuracy against the number of features d (1 to 9) for training sizes n = 20, 100, 1,000 and 10,000. This reproduces the Hughes peaking effect on our own data and justifies the final feature set.

II.6 Evaluation protocol

- **Deterministic replay.** His real-time tracker is replayed frame by frame. A YOLO result requested at frame i arrives at frame i + L. L = 1 matches his GPU (YOLO takes 11 ms; a frame lasts 33 ms); L = 3 imitates a slower onboard computer. On the USV, L is one 100 ms control step. The replay reproduces his reported YOLO detection rate of 83% on testv4.
- **Ground truth.** USV: the simulator's true leader box. Drone: Grounding DINO (2 prompts × horizontal flip = 4 opinions, fused with his own labelling rule, *without* YOLO as an opinion so the labels are independent of the model under test), checked by eye on contact sheets.
- **Training and test data are separate.** Drone: trained on one video, tested on the other. USV: runs split by random seed into two folds.
- **Measures.** Share of frames with IoU ≥ 0.5 against the true box; the same on the 20% of frames with the fastest vertical motion; wrong-target frames; jitter.

Part III. Preliminary results

III.1 Fast vertical motion breaks detection on the USV

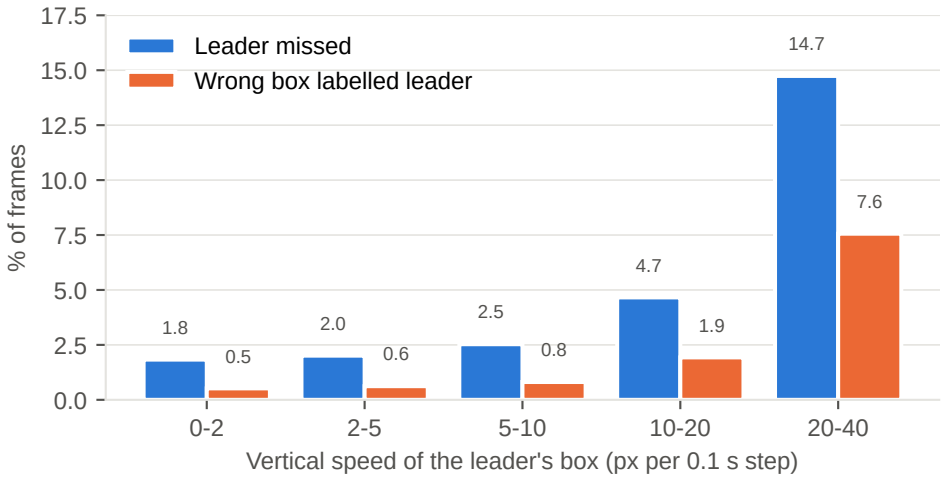


Figure 1. USV, weather-trained YOLO, 259,000 frames. Both error types rise steeply once the box moves more than 10 px per step. In huge waves the median vertical speed doubles (6 vs 3 px per step).

On the drone videos there is no clear trend: misses stay between 7% and 13% at every speed. Drone misses cluster on hard backgrounds instead, such as plants and the white tiled wall in IMG_7888. The per-frame speed is also noisy because the pseudo ground truth jitters. Smoothing and normalising by box size is planned (T2).

III.2 MLE of R: the drone code's noise model is wrong on the vertical axis



Figure 2. USV detection error variance (MLE) by YOLO confidence. Horizontal variance falls about 20× as confidence rises. Vertical variance barely changes, and the vertical error has a constant offset of about +0.09 box heights.

The drone code uses one variance for both axes, scaled by (0.5/confidence)². That roughly fits the horizontal axis but not the vertical one. On the USV, maximum likelihood also shows that optical flow's vertical error variance is about 4× YOLO's (0.045 vs 0.011 box sizes²). So on the boat, flow should carry less weight than YOLO.

III.3 MLE of Q: the likelihood peaks at 10× the drone setting on the USV

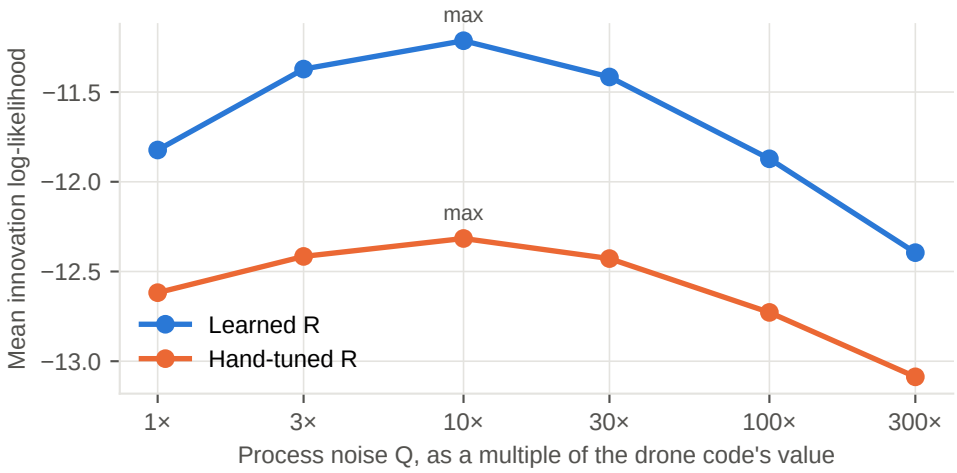


Figure 3. USV training fold 0: mean innovation log-likelihood for each candidate Q. Both folds and both R choices peak at 10×. Learned R also gives a higher likelihood than the hand-tuned R (−11.2 vs −12.3), so it describes the data better.

The peak is what the frame rates predict. The drone runs at 30 fps and the boat camera at 10 Hz, so the box moves about 3× as far per step on the boat, and the variance of that motion is about 9× larger. On the drone, the likelihood was still rising at the largest Q tried (100×), so the grid must be widened.

III.4 Tracking accuracy on the USV

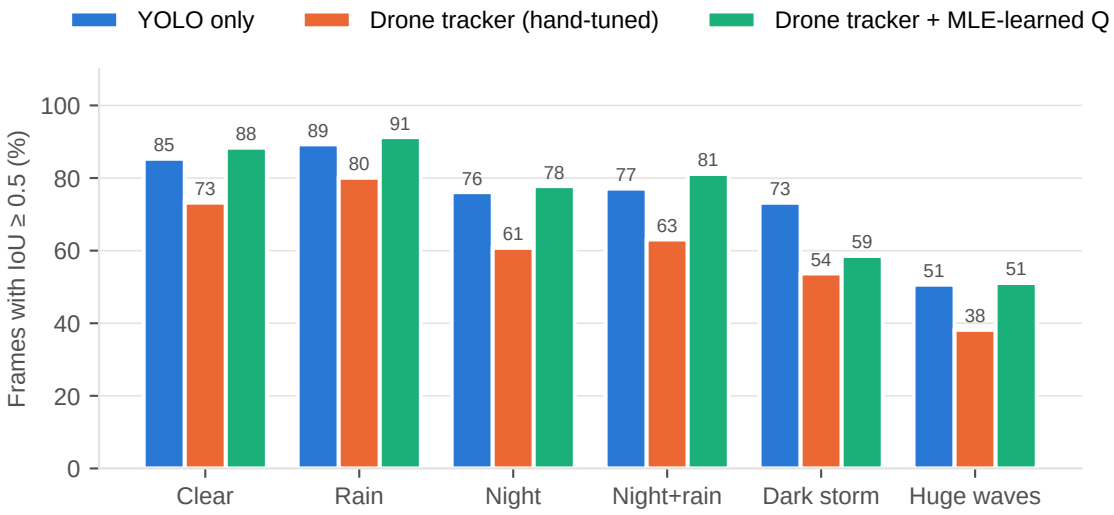


Figure 4. USV, 64 camera sequences, two folds split by seed. Frames tracked with IoU ≥ 0.5.

Condition	YOLO only	Drone tracker, hand-tuned	+ MLE-learned Q	+ MLE-learned R and Q	+ Bayes gate, learned R and Q
Clear	85.3 / 65.4	73.2 / 53.2	**88.4** / **81.2**	88.2 / 80.9	70.4 / 68.0
Rain	89.3 / 73.2	80.1 / 62.2	**91.3** / **83.0**	89.1 / 79.9	81.2 / 75.0
Night	76.1 / 38.9	60.7 / 28.1	**77.8** / **54.1**	75.1 / 53.8	68.0 / 50.7
Night + rain	77.1 / 44.9	63.1 / 32.5	**81.1** / **63.3**	77.8 / 60.9	70.8 / 58.4
Dark storm	**73.2** / **41.6**	53.7 / 26.6	58.5 / 38.8	53.7 / 38.1	44.7 / 33.4
Huge waves	50.6 / 9.5	38.2 / 7.8	**51.1** / **22.6**	46.7 / 19.0	36.9 / 21.1

Each cell: all frames / fastest 20% of frames, in % of frames with IoU ≥ 0.5. Bold = best in the row.

III.5 Tracking accuracy on the drone

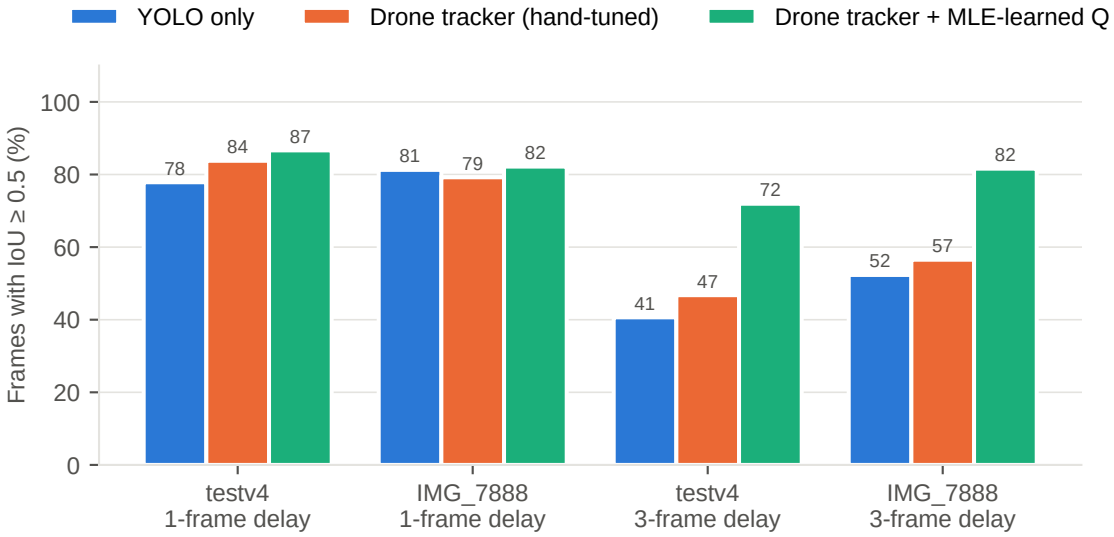


Figure 5. Drone, cross-video: trained on one video, tested on the other. Frames tracked with IoU ≥ 0.5.

Test video	YOLO only	Drone tracker, hand-tuned	+ MLE-learned Q	+ MLE-learned R and Q	+ Bayes gate, learned R and Q
testv4, 1-frame delay	77.9 / 68.0	83.8 / 79.6	86.6 / 86.4	**89.1** / 93.2	**89.1** / **96.1**
IMG_7888, 1-frame delay	81.3 / 63.4	79.2 / 77.7	82.2 / 84.6	82.4 / **86.3**	**88.2** / 82.3
testv4, 3-frame delay	40.6 / 4.9	46.8 / 24.3	**71.9** / **53.4**	47.3 / 35.0	48.1 / 41.7
IMG_7888, 3-frame delay	52.3 / 1.1	56.5 / 26.9	**81.6** / **70.3**	57.4 / 34.3	58.9 / 34.3

Each cell: all frames / fastest 20% of frames, in %. On IMG_7888 with a 1-frame delay, the Bayes gate cuts wrong-target frames from 16.0% (hand-tuned) to 7.2%.

III.6 Is the improvement real? Statistical tests and videos

USV. Each of the 64 camera sequences is one paired sample (hand-tuned and learned Q run on the same frames). The learned Q came from training runs that never include the test sequence. Wilcoxon signed-rank test on the per-sequence share of well-tracked frames:

Condition	Sequences	Learned Q better than hand-tuned	Median gain (points)	p	Learned Q better than YOLO only	Median gain (points)	p
All conditions	64	64/64	+13.7	4e-12	42/64	+1.9	0.11
Clear	12	12/12	+14.8	0.0005	11/12	+2.7	0.00098
Rain	8	8/8	+12.1	0.008	7/8	+1.7	0.016
Night	8	8/8	+17.0	0.008	6/8	+2.1	0.078
Night + rain	8	8/8	+18.5	0.008	8/8	+3.5	0.0078
Dark storm	8	8/8	+5.0	0.008	0/8	-14.8	0.0078
Huge waves	20	20/20	+12.1	2e-06	10/20	+0.3	0.7

Drone. Two videos, so we resample 1-second blocks of frames (block bootstrap, 4,000 resamples) to get a 95% confidence interval for the gain in the share of well-tracked frames:

Test video	Learned Q − hand-tuned, points [95% CI]	1-s blocks better / worse	Learned Q − YOLO only, points [95% CI]
testv4, 1-frame delay	+2.9 [-1.2, +7.3]	7 / 4 of 17	+8.8 [+4.1, +13.9]
IMG_7888, 1-frame delay	+3.0 [+1.1, +5.1]	9 / 0 of 29	+0.9 [-9.1, +9.3]
testv4, 3-frame delay	+25.2 [+13.3, +35.5]	13 / 2 of 17	+31.3 [+22.5, +41.6]
IMG_7888, 3-frame delay	+25.1 [+16.2, +35.9]	22 / 1 of 29	+29.3 [+18.7, +41.6]

> **Verdict.** Against the hand-tuned tracker, the learned Q is better on **every one of the 64 boat sequences** (p = 4e-12). On the drone it is better in all four cases, and in three the 95% interval excludes zero. With a slow detector the gain is about +25 points. Against raw YOLO, the learned tracker wins clearly in clear water, rain and night + rain, and when the drone's detector is slow. It is no better than YOLO in huge waves (p = 0.7) and **worse than YOLO in the dark storm** on all 8 sequences. That case is the main open problem for the final presentation.

Detection videos (folder `Experiments/PR_Course/rollouts/`). Each video shows three panels side by side: YOLO only, the hand-tuned drone tracker, and the drone tracker with learned Q. Green is the ground truth; each panel shows its current IoU and its running share of well-tracked frames.

File	What it shows
`drone_testv4_delay3.mp4`, `drone_img7888_delay3.mp4`	Drone with a slow detector (3-frame delay): the hand-tuned box lags behind fast climbs and drops; the learned-Q box stays on the drone.
`drone_testv4_delay1.mp4`, `drone_img7888_delay1.mp4`	Drone with the teammate's real delay: smaller gain (about +3 points).

USV video (cropped around the leader)	YOLO only	Hand-tuned	Learned Q
`usv_huge_waves_C10_huge_waves_s_dropout_5147_on_pv_Right.mp4`	61.8	41.2	65.8
`usv_clear_C8_clear_speed_s_dropout_5147_on_pv_Left.mp4`	86.8	68.6	89.1
`usv_dark_storm_L10_lap_dark_storm_5021_on_pv_Right.mp4`	75.9	55.4	60.7

The USV clips are the sequences with the largest gain in huge waves and in clear water, plus a typical (median-gain) dark-storm sequence, where learned Q improves on the hand-tuned tracker but stays below YOLO. The simulator's truth box sits on average about 0.1 box heights above YOLO's box in every condition. It is aligned in time (the best match is at a 0-step offset), and it lowers every tracker's IoU equally. In huge waves, single frames can show a larger mismatch.

III.7 What the results show

1. **Hand-tuned constants do not transfer.** The drone tracker, tuned on drone video, is worse than raw YOLO on the USV in every condition.
2. **MLE-learned Q is the one change that helps on both platforms.** On the USV it beats the hand-tuned tracker on all 64 sequences, beats raw YOLO in clear, rain, night and night + rain, ties with YOLO in huge waves, and loses to it in the dark storm. In the fastest huge-wave frames it more than doubles the share of well-tracked frames in the fastest huge-wave frames (9.5% → 22.6%). On the drone with a slow detector it raises accuracy from 47–57% to 72–82%.
3. **Learned R helps only in some cases.** It helps on the drone with a 1-frame delay, but hurts with a 3-frame delay and on the USV. The fit compares each box with ground truth at the frame the image was taken, but the filter uses the box L frames later. A latency-aware fit is planned.
4. **The Bayes gate depends on the platform.** On the drone, which has real distractors, it gives the best accuracy in fast frames (96% on testv4) and halves wrong locks on IMG_7888. On the USV, wrong leader boxes are rare (2.2% of candidates). Rejecting true boxes then causes drift, so the gate lowers accuracy. A loss-ratio sweep and target re-acquisition are planned.
5. **There is a trade-off between lag and smoothness.** The learned trackers follow the target more closely but jitter more (USV clear: 1.6 → 1.9 px; drone testv4: 9 → 21–25 px).

> **Limitations of these results.** There are only two drone videos, so the drone results come from two folds and are not significance-tested. Drone ground truth comes from Grounding DINO, not hand labels; frames where it was unsure (12% and 6%) are not scored. The USV results are simulation only and offline, with no closed-loop formation runs yet. Q was chosen from a grid.

Part IV. Plan for the final presentation

IV.1 Remaining experiments, in order of value per effort

#	Experiment	What it adds	Effort
1	Closed-form Q from ground truth (Method 2a) on both platforms, compared with the innovation-likelihood Q	The textbook MLE derivation on a slide; checks the grid result	Half a day
2	Wider Q grid on the drone; latency-aware R fit	Fixes the two known gaps in the current results	Half a day
3	Loss sweep $\lambda_{12}/\lambda_{21}$ = 0.1 to 20 with ROC curves; re-acquire the target after k rejections	Shows Ch. 2 decision theory directly; fixes the USV gate	1 day
4	PCA and 2-D Fisher LDA plots of the three classes; Hughes curve	Covers Ch. 3-2 and the dimensionality material	1 day
5	Q-Q plots of the errors; mixture model if they are not Gaussian	Tests the Gaussian assumption behind every method	Half a day
6	Drone: more test videos; smoothed motion analysis	Makes the drone result stronger than two folds	Drone member
7	Optional: learned Q inside the USV formation simulator (huge-wave laps, 3 seeds)	Effect on formation error, not just box accuracy	1–2 days

IV.2 Slide outline (about 15 minutes)

Slide	Content	Speaker
1–2	Two clips: a drone climbing fast and a boat camera in huge waves. Same failure: the box jumps vertically and YOLO misses it.	Both
3	The shared pipeline (Figure 0) and its hand-tuned constants	Drone
4–6	Theory: Gaussian MLE ($\hat{\Sigma}$ derivation) → R and Q; the minimum-risk rule as a likelihood-ratio inequality; Fisher LDA	USV
7	Data and ground truth; evaluation protocol	Both
8–9	USV: Figures 1–4 and the likelihood peak at 10×	USV
10–11	Drone: Figure 5, the Bayes gate in fast frames, side-by-side video	Drone
12	LDA projection and Hughes curve	USV
13	What transfers between platforms (learned parameters) and what does not (hand-tuned constants, optical flow on a 40 px boat)	Both
14	Limitations and future work; questions	Both

IV.3 Proposal talk on Nov 11 (3 minutes, 5 slides)

1. Problem: two platforms, one failure (video stills, Figure 1).
2. The pipeline and its hand-tuned constants (Figure 0).
3. Methods from the course: MLE of R and Q, the minimum-risk rule, PCA/LDA (Part II equations).
4. First result: hand-tuned against learned Q on both platforms (Figures 4 and 5).
5. Schedule and final deliverables (Part I, items 3 and 4).

Appendix: data and code

Code: `Tools/pr_course/` (USV), `Tools/pr_course/drone/` (drone replay, labelling, analysis), `Tools/pr_course/report/` (this report). Data and result files: `Experiments/PR_Course/`. The drone tracker is the teammate's `track.py`, ported with its constants unchanged.